Project architecture report

שמות הסטודנטים	ת.ז
קוד פרויקט	
מנחה	

- כל צוות נדרש למלא דוח זה. הדוח כולל בתוכו תשובות דוגמא המופיעות ב *italic* ונועדו להמחשת התשובות הנדרשות.
- את הדוח המלא יש לשלוח כתגובה *(reply)* למייל אישור הפרויקט (ולמנחה הפרויקט בלבד – ד"ר אריאל או אסי).
- הדוח המוחזר צריך להיות בפורמט *PDF*
- שם הקובץ צריך להכיל את מספר הפרויקט, לדוגמא ' לקבוצה מספר 20 שם הקובץ יהיה: *ProjectArchitecureDocument-20.docx*
- יש להגיש עד 15/01/2025.

Abstract

This project implements a Smart Home Automation System that allows users to remotely control and monitor devices via a mobile application. The system integrates IoT devices, cloud services, and microservices architecture to provide real-time data updates and control. The solution enhances convenience, security, and energy efficiency.

1. Introduction

The rapid growth of IoT has created opportunities for automating homes. This project explores the design and development of a Smart Home Automation System focusing on user-friendly interfaces and efficient data processing.

2. Problem Statement

Current home automation solutions often lack integration and customization options. This project addresses the challenge by developing a scalable and modular system tailored to user needs.

3. Objectives

1. *Develop a mobile app to control and monitor devices.*
2. *Integrate multiple IoT sensors and devices.*

3. Utilize cloud services for secure data storage.
4. Design an intuitive user interface.

4. User Stories

1. As a homeowner, I want to control my smart devices remotely, so I can ensure my house is secure when I'm not at home.
2. As a user, I want to receive real-time alerts from my sensors, so I can react quickly to any issues.
3. As a system administrator, I want to monitor device statuses centrally, so I can troubleshoot problems efficiently.

הצוות נדרש לכתוב בסעיף זה לפחות 10 Users Stories

Sample Test Code for User Stories

עבור לפחות 4 Users Stories הצוות יכתוב מבחני קוד כמפורט...

```
# User Story 1: Remote Device Control
import requests

def test_turn_on_device(device_id):
    response = requests.post(f"http://localhost:5000/api/device/{device_id}/turn_on")
    assert response.status_code == 200
    assert response.json()["status"] == "on"

test_turn_on_device(1234)

# User Story 2: Real-time Alerts
import websocket

def on_message(ws, message):
    print(f"Alert Received: {message}")

ws = websocket.WebSocketApp("ws://localhost:5000/alerts", on_message=on_message)
ws.run_forever()
```

5. Application Screenshots

Provide five screenshots of the application's design, created using Figma or other graphic tools. The screenshots should include key screens of the app, such as:
1. Home Screen
2. Device Control Screen
3. Sensor Data Dashboard

4. Settings Screen
5. User Profile or Login Screen.

Ensure the screenshots clearly showcase the user interface design, functionality, and navigation structure. Insert the images below:

[Placeholder for Screenshot 1]

[Placeholder for Screenshot 2]

[Placeholder for Screenshot 3]

[Placeholder for Screenshot 4]

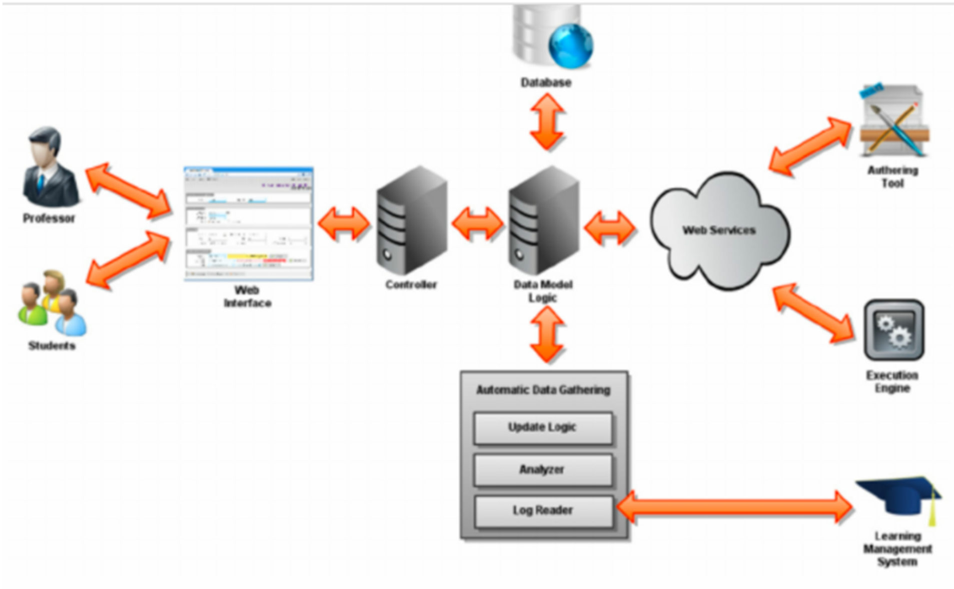
[Placeholder for Screenshot 5]

6. Typical User Flows

1. User Flow for Turning On a Smart Device:
- Open the mobile app   Select the device from the list   Tap 'Turn On'   Receive confirmation of the action.
2. User Flow for Viewing Sensor Data:
- Open the mobile app   Navigate to the 'Sensors' tab   Select a specific sensor   View real-time data and history.

7. Project Entity Diagram

The diagram below showcases key system components, including users, interfaces, and backend logic. This structure is inspired by the provided diagram and illustrates connections between clients, server, web interfaces, databases, and other logical components.



8. Implementation

8.1 Technologies Used

Backend: *Python, Flask*
Frontend: *Flutter*
Database: *PostgreSQL*
Cloud Services: *AWS IoT, Lambda Functions*